

1 Decidable languages

Languages that are always accepted or rejected by a Turing Machine (i.e., by a *decider* TM).

Definition We will use $\langle M \rangle$ to denote the binary encoding of a TM. Or, alternatively, write a program for M , and let $\langle M \rangle$ be its binary executable.

Examples:

Theorem 1.1 $A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w \}$ is decidable.

Proof We simply need to present a TM M_1 that decides A_{DFA} :

$M_1 =$ On input $\langle B, w \rangle$, where B is a DFA and w is a string

1. Simulate B on input w .
2. If the simulation ends in an accept state, *accept*. If it ends in a nonaccepting state, *reject*. ■

Theorem 1.2 $A_{NFA} = \{ \langle B, w \rangle \mid B \text{ is an NFA that accepts input string } w \}$ is decidable.

Proof We will use M_1 from above as a subroutine.

$M_2 =$ On input $\langle B, w \rangle$, where B is an NFA and w is a string

1. Convert NFA B to an equivalent DFA C .
2. Run TM M_1 on input $\langle C, w \rangle$.
3. If M_1 accepts, *accept*; otherwise, *reject*. ■

Theorem 1.3 $A_{REG} = \{ \langle R, w \rangle \mid R \text{ is a regular expression that generates string } w \}$ is decidable.

Proof $M_3 =$ On input $\langle R, w \rangle$, where R is a regular expression and w is a string

1. Convert regular expression R to an equivalent NFA A .
2. Run TM M_2 on input $\langle A, w \rangle$.
3. If M_2 accepts, *accept*; otherwise, *reject*. ■

Theorem 1.4 $E_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}$ is decidable.

Proof $M_4 =$ On input $\langle A \rangle$, where A is a DFA

1. Mark the start state of A .
2. Repeat until no new states get marked:
 - (a) Mark any state that has a transition coming into it from any state that is already marked.
3. If no accept state is marked, *accept*; otherwise, *reject*. ■

Theorem 1.5 $EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B) \}$ is decidable.

Proof $M_5 =$ On input $\langle A, B \rangle$, where A, B are DFAs

1. Construct a new DFA C from A and B , where C accepts only those strings that are accepted by either A or B , but not both. Thus, if A and B recognize the same language, C will accept nothing. The language of C is:

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Reminder: Regular languages are closed under complement, intersection, and union.

2. Run M_4 on input $\langle C \rangle$.
3. If M_4 accepts, *accept*; otherwise, *reject*. ■

Theorem 1.6 $A_{CFG} = \{ \langle G, w \rangle \mid G \text{ is a CFG that generates string } w \}$ is decidable.

Proof idea Convert to CNF, list all derivations with $2|w| - 1$ steps to see if any of them is equal to w

Theorem 1.7 $E_{CFG} = \{ \langle G \rangle \mid G \text{ is a CFG and } L(G) = \emptyset \}$ is decidable.

Proof idea Start with the terminal symbols in the grammar and mark any variable reachable in a bottom-up manner to see if the start symbol is reached.

But:

Theorem 1.8 $EQ_{CFG} = \{ \langle G, H \rangle \mid G \text{ and } H \text{ are CFGs and } L(G) = L(H) \}$ is **NOT** decidable.

2 Undecidability

- Problems that cannot be solved by computers (TMs).
- Practical significance: software testing.

Fact: Every TM can be encoded as a binary string according to some convention.

Example Assume:

- q_1 is the start state, q_2 is the accept state, q_3 is the reject state, and $q_i, i \geq 4$ are the rest of the states.
- $\Gamma = \{X_1, X_2, X_3, \dots\}$ where $X_1 = 0, X_2 = 1, X_3 = \sqcup, \dots$
- $\{L, R\} = \{D_1, D_2\}$
- A transition $\delta(q_i, X_j) = (q_k, X_l, D_m)$ is encoded as $0^i 10^j 10^k 10^l 10^m$.
- The TM is encoded as the list of its transitions separated by 11s since transition codes do not include two consecutive 1s. We also include a prefix of 1 to be able to convert to an integer:

$$TM = 1 \text{ Code1 } 11 \text{ Code2 } 11 \text{ Code3 } \dots$$

Then the i^{th} TM is the TM, where the binary representation of the integer i is the binary encoding of TM ($\langle M \rangle$). Some TMs do not make sense. For an i^{th} TM to be valid, there must be 5 blocks of 0s separated by single 1s between any pair of 11s. For example, the 3^{rd} TM does not make sense. For such TMs, we will assume that the language they accept is empty.

2.1 A language that is not Turing-recognizable

Imagine an infinite table, where:

- The rows: Every possible Turing Machine $M_0, M_1, M_2, \dots$ (since TMs can be represented as strings, we can list them in lexicographical order).
- The columns: Every possible input string $w_0, w_1, w_2, \dots$ (also listed lexicographically).

Each cell (i, j) in the table tells us whether machine M_i accepts string w_j .

TM i \ input string j	0 (w_0)	1 (w_1)	2 (w_2)	...	i (w_i)	...	k (w_k)	...
0	0	0	0		0		1	
1	0	0	0		1		1	
2	0	0	0		0		1	
...								
$i = \langle M \rangle$	0	1	0		1		0	
...								
$k = \langle N \rangle ?$	1	1	1		0		?	
...								

The diagonal will be defined as

$$D = a_1 a_2 a_3 \dots$$

where $a_i = 1$ if $(i, i) = 0$ and $a_i = 0$ if $(i, i) = 1$.

Define:

$$L_d = \{ \langle M \rangle \mid \langle M \rangle \notin L(M) \}$$

Assume that L_d is recognizable by some TM N (defined in row k of the table); i.e., $L_d = L(N)$. Now, the question becomes: is $\langle N \rangle = w_k \in L_d$?

- If $w_k \in L_d$: By the definition of L_d , N must not accept w_k . But if N recognizes L_d , it must accept w_k . (Contradiction)
- If $w_k \notin L_d$: By definition, N must accept w_k . But if N accepts w_k , then by the definition of L_d , w_k should be in L_d . (Contradiction)

Therefore, no such N is possible: L_d is not recognizable.

2.2 Recognizable but not decidable language

Define:

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM, and } w \in L(M) \}$$

Assume A_{TM} is recognizable by a TM U (the “universal” TM): U on $\langle M, w \rangle$

- Check if M is a valid TM encoding.
- If so, evaluate M on w .
- If M accepts w , *accept*.
- If M rejects w , *reject*.

Note that U is not a decider. If M loops on w , U will also loop. Then $A_{TM} = L(U)$.

Theorem 2.1 A_{TM} is not decidable.

Proof by contradiction. Assume that A_{TM} is decided by some TM H :

$$H(\langle M, w \rangle) = \begin{cases} \text{accept, if } w \in L(M) \\ \text{reject, otherwise} \end{cases}$$

From H , we can construct another TM D that decides L_d : D on $\langle M \rangle$: Is $\langle M \rangle \in L_d$?

- Run H on $\langle M, \langle M \rangle \rangle$.
- Accept if H rejects.
- Reject if H accepts.

We know that no such D exists since L_d is not recognizable (hence, not decidable). Thus, no such H exists. A_{TM} is not decidable. ■

Definition A language A is co-Turing recognizable if its complementation, $\bar{A}$ is Turing recognizable.

Theorem 2.2 A language A is decidable iff it is Turing recognizable and co-Turing recognizable.

Proof There are two parts of this proof:

- If A is decidable, then A is Turing recognizable. Also $\bar{A}$ is Turing recognizable (simply switch q_A and q_R).
- If both A and $\bar{A}$ are Turing recognizable, then we can build M_1 for A , and M_2 for $\bar{A}$. Then, assume M is a decider for A . M can run M_1 and M_2 in parallel. Note that, $\forall w$, either $w \in A$, or $w \in \bar{A}$. Then, if M_1 accepts w , M will accept. Otherwise, if M_2 accepts w , then M will reject.

Corollary 2.3 $\overline{A_{TM}}$ is not Turing-recognizable.

Proof We know that A_{TM} is Turing-recognizable. If $\overline{A_{TM}}$ were also Turing-recognizable, A_{TM} would be decidable. But we know that A_{TM} is undecidable, so $\overline{A_{TM}}$ must not be Turing-recognizable.